

2025-2026 学年面向对象程序设计模拟试卷 01-参考答案

一、选择题

1.

答案: A

id 是常量成员, ref 是引用成员, 它们都必须在初始化列表中完成初始化, 不能先默认生成再在构造函数体内赋值。同时, ref 必须绑定到一个真正存在的 `std::string` 对象上, 因此参数也应当是引用类型。综合来看, 只有 A 正确。

2.

答案: B

逐组分析:

1. (a) 不能构成重载。按值传参时, 顶层 `const` 不属于函数参数类型区分的一部分, 所以这两句本质上是同一个函数。
2. (b) 可以构成重载。`int&` 和 `const int&` 的参数类型不同。
3. (c) 也可以构成重载。`int*` 和 `const int*` 是不同类型。

因此正确答案是 (b) 和 (c)。

3.

答案: B

程序输出:

2 3 3

原因如下:

1. `Base b = d;` 发生对象切片, `b` 已经是一个独立的 `Base` 对象, 因此 `b.g()` 调用的是 `Base::f(1)`, 结果为 2
2. `r` 是指向 `Derived` 对象的基类引用, `r.g()` 中的 `f()` 会发生动态绑定, 调用 `Derived::f`; 但默认参数取决于静态类型, 在 `Base::g()` 中写的是 `f()`, 因此使用的是基类版本的默认参数 1, 结果为 $1 + 2 = 3$
3. `d.g()` 继承来的仍然是 `Base::g()`, 其中调用虚函数时依旧会动态绑定到 `Derived::f(1)`, 所以也是 3

这题同时考察了对象切片、虚函数动态绑定和默认参数绑定时机。

4.

答案: D

`list` 的迭代器不是随机访问迭代器, 而 `std::sort` 需要随机访问迭代器, 因此不能直接写成 `std::sort(list.begin(), list.end())`。

如果要排序 `list`, 应该使用它自己的成员函数 `list.sort()`。

5.

答案: C

`explicit` 的作用, 是阻止某些隐式类型转换。例如:

```
A a = 3;
```

通常会被禁止; 但下面这种直接初始化仍然是允许的:

```
A a(3);
```

因此 C 错。

另外, 把 `operator+` 写成非成员函数, 确实更有利于左右两边都进行自动类型转换, 这是常见设计经验。

6.

答案: B

若一个类内部有裸指针成员, 并在构造函数中使用 `new` 分配动态资源, 那么编译器默认合成的复制构造函数和赋值运算符往往只做“按成员逐个复制”, 也就是把指针地址直接复制过去。这样多个对象就会指向同一块动态内存, 最终可能产生重复释放、悬挂指针等问题。

因此 B 是正确的。这一题本质上考察的是资源类的复制控制, 也就是常说的“三件套”问题。

7.

答案: C

`weak_ptr` 不是普通裸指针, 它不能直接解引用, 也不会拥有对象。

当最后一个 `shared_ptr` 销毁后, 被管理对象就不存在了, 这时 `weak_ptr` 只能通过 `expired()` 判断对象是否还活着, 或者通过 `lock()` 尝试获取一个新的 `shared_ptr`。

因此 C 错。

8.

答案: C

抽象类不能直接定义对象, 但完全可以定义该类型的指针或引用, 用来指向派生类对象, 这是多态最常见的使用方式。

A 错, 因为“有虚函数”不等于“抽象类”; 只有存在纯虚函数时才一定是抽象类。

B 错, 抽象类当然可以有构造函数。

D 也错, 纯虚函数虽然是纯虚的, 但在某些场景下基类中仍然可以给出函数定义。

二、程序阅读题

1.

输出结果:

A2 B3 A5 C5 C3 | C2

执行过程如下：

1. Trace a(2); 调用普通构造函数，输出 A2
2. Trace b = a; 调用复制构造函数， $b.v = a.v + 1 = 3$ ，输出 B3
3. Trace c(5); 调用普通构造函数，输出 A5
4. 离开内部作用域时，后构造的先析构，因此先析构 c，输出 C5
5. 再析构 b，输出 C3
6. 之后执行 `cout << "| "`;
7. main 结束时再析构 a，输出 C2

这题主要考察构造、复制构造和析构的调用顺序。

2.

输出结果：

33 3

twice 的参数是按值传递，因此调用 twice(b) 时，函数内部的 x 是 b 的一个副本。

函数体：

```
return x += x;
```

会先执行 $x += x$ 。对 Box 来说，operator+= 的逻辑是：

```
v = v * 10 + rhs.v;
```

因此当 x 的值为 3 时：

```
x += x -> 3 * 10 + 3 = 33
```

返回值打印为 33。

而原来的 b 从头到尾没有被修改，所以第二个输出仍是 3。

3.

输出结果：

9 9 1 3 5

需要注意这一句：

```
vector<int> w(2, 9);
```

它不是“预留两个位置”，而是直接创建了两个元素，值都为 9。

随后：

```
remove_copy_if(v.begin(), v.end(), back_inserter(w),  
               [](int x) { return x % 2 == 0; });
```

会把 v 中所有“不满足删除条件”的元素复制到 w 末尾。
删除条件是“偶数”，所以被保留下来的就是奇数 1、3、5。

因此 w 最终内容为：

9 9 1 3 5

这题考察的重点是：back_inserter 是在原容器末尾继续追加，不会覆盖原来已经存在的元素。

4.

输出结果：

+1 2 1 -1 E

执行过程如下：

1. shared_ptr<Node> p(new Node(1)); 创建对象，构造函数输出 +1
2. shared_ptr<Node> q = p; 后，引用计数变为 2，因此输出 2
3. 内层作用域结束时，q 被销毁，但 p 仍存在，因此计数变回 1，输出 1
4. 外层作用域结束时，p 也被销毁，对象引用计数降为 0，真正释放，析构函数输出 -1
5. 这时 wp 观察的对象已经不存在了，所以 wp.expired() 为真，输出 E

这题主要考察 shared_ptr 的引用计数和 weak_ptr 的失效判断。

三、程序填空题

1.

填空答案：

```
(1) rhs.sz ? new int[rhs.sz] : 0
(2) std::copy(rhs.data, rhs.data + sz, data)
(3) rhs.sz ? new int[rhs.sz] : 0
(4) std::copy(rhs.data, rhs.data + sz, data)
(5) delete[] data
```

说明如下：

1. 复制构造函数要重新申请一块与源对象同样大小的动态数组
2. 然后把源对象中的数据逐个复制过来，这样才是深拷贝
3. 赋值运算符中同样要重新申请新空间，再复制内容
4. 析构函数负责释放动态数组

这题本质上就是资源类的复制控制。

2.

填空答案：

```
(6) override
(7) new Circle(*this)
(8) s.clone()
```

说明如下：

1. Circle::area() 是对基类纯虚函数的重写，因此可写 override
2. clone() 的作用是返回当前对象的一份同类型复制，所以应写 new Circle(*this)
3. 外部只拿到一个 Shape& s 时，并不知道其真实派生类型，因此应通过虚函数 clone() 完成多态复制

这正是“基类指针或引用 + clone 虚函数”这一类写法的核心思想。

四、编程题

1. 模板函数 merge_unique

这一题要求写一个泛型合并算法，难点不在“合并有序区间”本身，而在于还要**去掉重复值**，而且重复可能来自三种情况：

1. 同一个区间内部重复
2. 另一个区间内部重复
3. 两个区间之间有相同值

因此，写法上通常要一边比较两个区间当前元素，一边记录“上一次真正输出的值”。只要当前候选值与上一次输出值相同，就跳过；否则才写入目标位置。

参考实现如下：

```
#include <iterator>

template <class In1, class In2, class Out>
Out merge_unique(In1 first1, In1 last1, In2 first2, In2 last2, Out dest)
{
    typedef typename std::iterator_traits<In1>::value_type value_type;

    bool has_last = false;
    value_type last = value_type();

    while (first1 != last1 && first2 != last2) {
        value_type current;

        if (*first1 < *first2) {
            current = *first1;
            ++first1;
        } else if (*first2 < *first1) {
            current = *first2;
            ++first2;
        } else {
```

```

        current = *first1;
        ++first1;
        ++first2;
    }

    if (!has_last || !(current == last)) {
        *dest++ = current;
        last = current;
        has_last = true;
    }
}

while (first1 != last1) {
    value_type current = *first1;
    ++first1;
    if (!has_last || !(current == last)) {
        *dest++ = current;
        last = current;
        has_last = true;
    }
}

while (first2 != last2) {
    value_type current = *first2;
    ++first2;
    if (!has_last || !(current == last)) {
        *dest++ = current;
        last = current;
        has_last = true;
    }
}

return dest;
}

```

对于测试数据：

a: 1 2 2 4 7 9

b: 2 3 4 4 8 9

按非降序合并并去重后，结果自然就是：

1 2 3 4 7 8 9

这题考察的是模板函数、迭代器、输出迭代器以及泛型算法的组织方式，属于比较像考试风格的一类大题。

2. Shape、Rectangle、Circle 和 Picture

这一题最核心的考点有四层：

1. 用抽象基类表达 “所有图形的共同接口”
2. 用派生类分别实现矩形和圆
3. 用基类指针容器统一管理不同类型对象
4. 让 Picture 正确支持深拷贝，保证复制后的对象彼此独立

其中最关键的是 clone()。因为 Picture 保存的是 Shape*，如果只知道传入参数是 const Shape&，就无法直接写出 “复制真实派生类对象” 的代码。这时必须让每个派生类自己实现 clone()，由它返回一份同类型副本。

参考实现如下：

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Shape {
public:
    Shape(const string& n = "") : name(n) {}
    virtual ~Shape() {}

    virtual double area() const = 0;
    virtual void scale(double factor) = 0;
    virtual Shape* clone() const = 0;

    virtual void print(ostream& os) const {
        os << name << " " << area();
    }

protected:
    string name;
};

class Rectangle : public Shape {
public:
    Rectangle(const string& n = "", double w = 0, double h = 0)
        : Shape(n), width(w), height(h) {}

    double area() const {
        return width * height;
    }
}
```

```

void scale(double factor) {
    width *= factor;
    height *= factor;
}

Shape* clone() const {
    return new Rectangle(*this);
}

private:
    double width;
    double height;
};

class Circle : public Shape {
public:
    Circle(const string& n = "", double r = 0)
        : Shape(n), radius(r) {}

    double area() const {
        return 3.14 * radius * radius;
    }

    void scale(double factor) {
        radius *= factor;
    }

    Shape* clone() const {
        return new Circle(*this);
    }

private:
    double radius;
};

class Picture {
public:
    Picture() {}

    Picture(const Picture& rhs) {
        for (std::size_t i = 0; i != rhs.shapes.size(); ++i)
            shapes.push_back(rhs.shapes[i]->clone());
    }

```



```
}
```

```
Picture& operator=(const Picture& rhs) {  
    if (this != &rhs) {  
        clear();  
        for (std::size_t i = 0; i != rhs.shapes.size(); ++i)  
            shapes.push_back(rhs.shapes[i]->clone());  
    }  
    return *this;  
}
```

```
~Picture() {  
    clear();  
}
```

```
void add(const Shape& s) {  
    shapes.push_back(s.clone());  
}
```

```
void scale_all(double factor) {  
    for (std::size_t i = 0; i != shapes.size(); ++i)  
        shapes[i]->scale(factor);  
}
```

```
double total_area() const {  
    double sum = 0;  
    for (std::size_t i = 0; i != shapes.size(); ++i)  
        sum += shapes[i]->area();  
    return sum;  
}
```

```
friend ostream& operator<<(ostream& os, const Picture& p) {  
    for (std::size_t i = 0; i != p.shapes.size(); ++i) {  
        p.shapes[i]->print(os);  
        os << endl;  
    }  
    return os;  
}
```

private:

```
vector<Shape*> shapes;
```

```

void clear() {
    for (std::size_t i = 0; i != shapes.size(); ++i)
        delete shapes[i];
    shapes.clear();
}
};

```

下面核对题目中的输出：

1. p 中有一个 3 x 4 的矩形和一个半径为 2 的圆
2. p.total_area() 为

$$3 * 4 + 3.14 * 2 * 2 = 12 + 12.56 = 24.56$$
3. Picture q = p; 之后, q 得到的是深拷贝
4. q.scale_all(2); 会让矩形边长和圆半径都乘以 2
5. 此时 q 中矩形面积变为 48.00, 圆面积变为 50.24

因此：

$q.total_area() = 48.00 + 50.24 = 98.24$

并且输出 q 时会逐行打印：

RectA 48.00

CircleA 50.24

这题如果能够完整写出来，说明抽象类、虚函数、多态、深拷贝和资源管理这几个关键点已经比较扎实了。